TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

Week 12

Pull requests — Module 04 checkpoint

Phase 2 — Building Real Apps · Weeks 7–12 · Modules 03–04

No D2L login required. This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

In this packet

1. 04.10 Pull Requests
2. 04.11 Final Checkpoint

Learning objective: Push a feature branch to GitHub, open a pull request, leave and respond to inline review comments, merge the PR with a merge commit, and sync the result to the local `main`.

Lesson 10: Pull Requests

By the end, you will have opened a pull request on GitHub, reviewed and merged it — and opened a PR to the class practice repo plus reviewed a classmate's PR.

~35 minutes of typing + clicking

> A **pull request** (PR) is a proposal to merge one branch into another. The web UI shows the diff, lets people comment, and gates the merge. Every team uses this workflow.

Part 1: a PR on your own repo

You'll walk the full loop solo first — author and reviewer both — so the mechanics are familiar before the class exercise.

Step 1: create a feature branch

1. From your project directory.

```
cd ~/docker-course/fastapi-app
```

2. Confirm clean.

```
git status
```

3. Make sure you're up to date with the remote.

```
git pull
```

4. Create a feature branch.

```
git switch -c add-version-route
```

Step 2: build the feature

5. Open `main.py` in VS Code. Above `app.include_router(items.router)`, add a new route:

```
@app.get("/version")
def version():
    return {"version": "0.1.0"}
```

6. Save.

7. Commit.

```
git commit -am "Add /version route returning 0.1.0"
```

8. Edit `NOTES.md` to add a line: `Added /version`. Save and commit.

```
git commit -am "Note /version in NOTES"
```

Step 3: push the branch

9. Push the branch — `-u` because it's the first push of this branch.

```
git push -u origin add-version-route
```

You should see:

```
remote:
remote: Create a pull request for 'add-version-route' on GitHub by visiting:
remote:      https://github.com/<username>/fastapi-app/pull/new/add-version-route
remote:
To github.com:<username>/fastapi-app.git
 * [new branch]      add-version-route -> add-version-route
branch 'add-version-route' set up to track 'origin/add-version-route'.
```

Note the URL GitHub suggested. That's the PR creation page.

Step 4: open the pull request

10. Open the URL from step 9 in your browser. (Or go to your repo on github.com — a yellow **"Compare & pull request"** banner appears for recently pushed branches; click it. If the banner's gone, use **Pull requests → New pull request**.)

11. You should see:

- **base:** `main`
- **compare:** `add-version-route`
- The diff of your two commits

12. Set the title to something useful: `Add /version route`.

13. Set the description to something like:

```
Adds GET /version returning {"version": "0.1.0"}.

- New route in main.py
- Note in NOTES.md
```

14. Click **Create pull request**.

15. You're on the PR page. Tour it:

- **Conversation tab** — comments, status events.
- **Commits tab** — your two commits.
- **Files changed tab** — the diff.

Step 5: review

16. You'll review your own PR — yes, it's a little weird. The real peer review comes in Part 2; this round is to learn where the buttons are.

17. Click **Files changed**. Hover over a line in the diff — a blue **+** icon appears at the left edge. Click it to add an inline comment.

18. Leave one comment on a line of `main.py`. For example, on the `return {"version": "0.1.0"}` line, add:

```
Should we read the version from a constant or env var instead of hardcoding?
```

19. Click **Add single comment** (or **Start a review** to batch comments and submit them all at once).

20. Go back to the **Conversation** tab. Your comment should appear inline with the diff snippet.

Step 6: respond to feedback

21. As the author, address the comment. Edit `main.py` — change the version line:

```
APP_VERSION = "0.1.0"


@app.get("/version")
def version():
    return {"version": APP_VERSION}
```

(`APP_VERSION` constant declared above the route.)

22. Save, commit, push.

```
git commit -am "Hoist version to APP_VERSION constant"
git push
```

23. Refresh the PR page on GitHub. The new commit shows up in the **Commits** tab and in the diff — a PR tracks its branch, so pushing updates the PR automatically.

24. Reply to the original review comment with something like `Done — moved to APP_VERSION`. Click **Comment**.

25. Mark the thread resolved (the **Resolve conversation** button on the comment thread).

Step 7: merge

26. Scroll to the bottom of the **Conversation** tab. There's a green **Merge pull request** button.

27. Pick a merge style — the dropdown arrow on the button offers three:

- **Create a merge commit** — keeps the branch's full history with a merge commit on top.
- **Squash and merge** — collapses all PR commits into one on `main`. Common for small features.
- **Rebase and merge** — replays the branch's commits on top of `main` linearly (no merge commit). For this drill, pick **Create a merge commit**.

28. Click **Merge pull request**, then **Confirm merge**.

29. GitHub shows "Pull request successfully merged and closed." There's a **Delete branch** button — click it. (The merged branch is no longer needed.)

Step 8: sync locally

30. Back in the terminal.

```
cd ~/docker-course/fastapi-app
git switch main
git pull
```

You should see the merge commit + your branch's commits arrive.

31. Look at the graph.

```
git log --oneline --graph -8
```

The merge from your PR is there.

32. The branch is gone on the remote. Delete the local copy too.

```
git branch -d add-version-route
```

33. Confirm.

```
git branch
```

Only `* main`.

34. Test the new route end-to-end. Start your stack if it's not running.

```
docker compose up -d
sleep 2
curl http://localhost:8000/version
```

```
{"version":"0.1.0"}
```

Part 2: the class PR exercise (fork → PR → review)

Your instructor hosts a hub repo: `<INSTRUCTOR-GITHUB-USERNAME>/course-pr-practice`. You **can't push to it** — you don't own it. So you **fork** it (a fork is your own server-side copy on GitHub), push to your fork, and open a PR from your fork back to the instructor's repo. This is exactly how open-source contributions work.

35. In your browser, open:

```
https://github.com/<INSTRUCTOR-GITHUB-USERNAME>/course-pr-practice
```

Click **Fork** (top-right) → **Create fork**. GitHub drops you on your copy: `<username>/course-pr-practice`.

36. Clone **your fork** (not the instructor's repo).

```
cd ~/docker-course
git clone git@github.com:<username>/course-pr-practice.git
cd course-pr-practice
```

37. Create a branch for your card.

```
git switch -c add-student-card
```

38. Add your student card. Create `students/<your-github-username>.md` in VS Code (the `students/` folder already exists in the repo) with content like:

```
# Your Name

- GitHub: @<your-github-username>
- Favorite Git command so far: `git log --oneline --graph`
```

39. Stage, commit, push.

```
git add students/<your-github-username>.md
git commit -m "Add student card for <your-github-username>"
git push -u origin add-student-card
```

40. Open the PR. Go to your fork on github.com and click the **"Compare & pull request"** banner. Check the header carefully:

- **base repository:** `<INSTRUCTOR-GITHUB-USERNAME>/course-pr-practice`, **base:** `main`
- **head repository:** `<username>/course-pr-practice`, **compare:** `add-student-card`

This is a **cross-fork PR** — proposing changes into a repo you can't push to.

41. Title: `Add student card: <your-github-username>`. Click **Create pull request**. (You're opening it on the **instructor's** repo — check the URL.)

42. Now review a classmate. Go to `<INSTRUCTOR-GITHUB-USERNAME>/course-pr-practice` → **Pull requests** tab. Pick an open PR from a classmate (your instructor may assign who reviews whom — a simple rule: review the PR opened right after yours in the list).

43. Open their PR → **Files changed** → leave **at least one inline comment**. Make it real — a question, a genuine remark, or a suggestion, not just "LGTM".

44. Click **Review changes** (top-right of Files changed) → select **Comment** (or **Approve**) → **Submit review**.

45. The instructor merges card PRs during or after class. When yours merges, you'll get a notification, and your card will appear in the hub's `students/` folder alongside everyone else's.

46. Log it.

```
echo "First PR merged on GitHub: /version route; card PR opened on course-pr-practice" >> ~/docker-course/notes/notes.txt
```

Watch out

- **Never commit directly to `main`** when collaborating. Branch + PR is the workflow. Direct commits to `main` skip review.
- **Fork vs clone.** A **fork** is your copy of someone else's repo *on GitHub* (server-side). A **clone** is a local copy of any repo (your machine). For the class exercise you do both: fork the hub, then clone your fork.
- **The "Compare & pull request" banner is temporary.** It only shows for recently pushed branches. If it's gone, **Pull requests** → **New pull request** does the same thing.
- **A merged PR's branch should be deleted.** Both on GitHub (the **Delete branch** button) and locally (`git branch -d`). Stale branches pile up fast otherwise.
- **Squash vs merge commit vs rebase** is a team-by-team religious decision. Pick one and be consistent. The lessons use merge commits because they're the most explicit.

Checkpoint

```
git -C ~/docker-course/fastapi-app log --oneline --graph -5
```

Should show the merge of `add-version-route`.

```
git -C ~/docker-course/fastapi-app branch
```

Should show only `* main`.

```
curl http://localhost:8000/version
```

Should print `{"version":"0.1.0"}`.

On GitHub:

- Your `fastapi-app` repo's **Pull requests** tab should show one **Merged** PR with your description and the conversation.
- `<INSTRUCTOR-GITHUB-USERNAME>/course-pr-practice` should show your card PR (**Open**, or **Merged** once the instructor gets to it).
- A classmate's PR on the practice repo should have at least one review comment from you.

Next up: 11-final-checkpoint.md — The whole loop, from memory

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Complete a full branch, commit, push, pull request, review, and merge cycle from memory and verify the new `/info` route works end to end.

Lesson 11: Final Checkpoint

Time: ~20 min — **no instructor help.**

This is the cumulative drill. You'll do a complete branch → commit → push → PR → review → merge cycle from memory. If you forget a command, reread the relevant lesson — but don't ask the instructor.

The task

Add a small feature to your FastAPI app and ship it through a real PR workflow.

Feature spec: add a `GET /info` route that returns:

```
{
  "app": "fastapi-app",
  "version": "0.1.0",
  "owner": "<your-github-username>"
}
```

The values can be hardcoded — this is a Git workflow drill, not an API drill.

The steps (in order)

1. From `~/docker-course/fastapi-app`, confirm clean state and pull latest.

```
cd ~/docker-course/fastapi-app
git status
git pull
```

2. Create a feature branch named `add-info-route`.

```
git switch -c add-info-route
```

3. Implement the feature:

- Edit `main.py` — add the `/info` route.
- Edit `NOTES.md` — add a line `Added /info`.

4. Stage and commit each file in a **separate commit** with descriptive messages:

- Commit 1: `Add /info route`
- Commit 2: `Note /info in NOTES.md`

5. Confirm the two commits.

```
git log --oneline -3
```

6. Push the branch.

```
git push -u origin add-info-route
```

7. On GitHub, open a pull request from `add-info-route` into `main` (the **"Compare & pull request"** banner, or **Pull requests** → **New pull request**). Title: `Add /info route`. Description: a short summary.

8. Leave at least one inline comment on your own PR (or ask a classmate to review it — you know where their repos are now). Reply to it.

9. Merge the PR (use **Create a merge commit**). Delete the branch on GitHub.

10. Pull on your local `main`.

```
git switch main
git pull
git branch -d add-info-route
```

11. Verify the route is live.

```
docker compose up -d
sleep 2
curl http://localhost:8000/info
```

Expected:

```
{"app":"fastapi-app","version":"0.1.0","owner":"<your-github-username>"}
```

The acceptance test

Each of these must succeed:

```
git status
```

Clean.

```
git log --oneline -1
```

Shows the merge commit from your PR.

```
git branch
```

Shows only `* main`.

```
curl -s http://localhost:8000/info | python -m json.tool
```

Prints your three-key JSON.

On GitHub:

- The **Pull requests** tab shows your PR as **Merged**.
- The commit history on `main` shows your two feature commits + the merge commit.

What to submit

A screenshot showing **all four** of these in one image:

1. Your terminal with `curl http://localhost:8000/info` and its JSON response.
2. Your terminal with `git log --oneline --graph -5`.
3. Your GitHub PR page showing **Merged** status.
4. Your GitHub repo file view of `main.py` with the `/info` route visible.

Send it to the instructor however they've asked.

Then:

```
echo "Module 04 final checkpoint: passed $(date -Iseconds)" >> ~/docker-
course/notes/notes.txt
```

Self-grade

You pass this module if:

- [ ] The PR was opened, reviewed (with at least one comment), and merged.
- [ ] The merged branch was deleted on both GitHub and locally.
- [ ] `curl http://localhost:8000/info` returns the expected JSON.
- [ ] The merge commit shows in `git log --oneline --graph` on `main`.

What's next

Module 05 turns on Continue.dev's AI features in VS Code — autocomplete, chat, and agent mode — using the repo you just built as the working codebase. Every change from here on flows through the same branch + PR + merge cycle you just drilled.

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.